

TÖL105M Tölvugrafík

Verkefni 2

Ólafur Kristófer Helgaso – okh4@hi.is

Linkur á forritið: <https://olafurkristofer.github.io/TOL105M-tolvugrafik/verkefni2/viewpoints.html>

Verkefnið byrjaði á því að skoða hvernig upprunalega viewpoints forritið virkaði, hvernig allt var teiknað og hvernig hreyfingin var útfærð. Þegar ég var búinn að kynna mér forritið ákvað ég á að byrja á því að bæta við þaki á öll húsin, ég bjó til nýja hornpunkta til að nota í þakinu af því að ég vildi teikna þakið úr 6 þríhyrningum, tveir og tveir lagðir saman mynda löngu hliðina og svo tveir á endunum til að loka þakinu. Þegar það var komið gerði ég þakið aðeins stærra en húsið og lagði það í raun ofan í húsið til að fela allt tóma plássið sem var inn í þakinu og þannig að þakið myndi sitja aðeins fyrir utan húsið sjálft. Myndin til hægri sýnir hvernig þakið kom út. Þegar það var komið bætti ég við fleiri gerðum af húsum, ég notaði upprunalega house fallið og breytti því til að teikna blokk með stillanlegum hæðum og einnig hús með bílskúr tengdan við, sem heitir L house í kóðanum. Eftir að ég var búinn bæta við nokkrum húsum fór ég í að setja inn brú. Ég ákvað að búa til frekar einfalda hönnun, teningar sitthvoru megin við veginn sem sitja ofan á hvorum öðrum og hver liggur aðeins innar en sá sem er undir honum og á endanum mætast þeir í miðjunni. Útfærslan var frekar einföld, ég bjó til einn tening í stærð sem ég var sáttur með og staðsetti hann aðeins fyrir utan veginn síðan teiknaði ég



hann aftur og aftur en færði hann aðeins innar með `translate(0, 0.25, 0)` og `translate(0, -0.25, 0)` ég gerði þetta 8 sinnum báðum megin við veginn þannig lokaútkoman er í stigi yfir veginn. Í heildina eru þetta þá 16 tengingar 8 sitthvoru megin sem ég

litaði síðan í mismunandi litum til að sýna betur hvern tenging eða hvert þrep.

Þessar fyrstu tvær viðbætur voru frekar þæginlegar þar sem þetta eru bara hlutir sem eru teiknaðir inn og svo ekkert gert við þá, næst fór ég í að bæta við öðrum bíl sem var aðeins erfiðara en á endum var lausnin frekar einföld. Ég bætti við breytum fyrir nýjan bíl og notaði sömu útreikninga sem upprunalegi bílinn notar nema ég mínusa 0.5 og tel frá

360 niður í 0 til að bílinn fari í öfuga átt. Einnig færði ég bílana aðeins til þannig að þeir myndi keyra sitthvoru megin á akreyninni. Hér er nýja útfærslan á keyrslu bílanna. Til að klára veginn bætti ég við hvítu línu á

vegin til að aðskilja bílana betur, það gerði ég með því að annan hvern TRACK_PTS og reikna nýja hornpunkta út frá miðjunni á veginum, bætti þeim síðan í nýtt fylki og teiknaði það fylki í drawScenery fallinu. Síðan kom mouse look viðbætingin, ég setti upp breytur fyrir staðsetninguna á notandanum og einnig hvert hann væri að horfa, eftir það setti ég inn eventListener fyrir „mousemove“ sem virkar bara ef view=0 þar inni breytti ég gildinu á hvert notandinn var að horfa með því að mínusa hreyfinguna á X og Y ásnum sinnum 0.002. Þetta eru línurnar sem útfæra það: yaw -= e.movementX * 0.002; og pitch -= e.movementY * 0.002;. Einnig bætti ég við hámarks snúning sem notandinn get gert til að reyna líkja eftir raunveruleikanum. Svo notaði ég hvert notandinn var að horfa samkvæmt X ásnum til að útfæra hreyfinguna sem sést hér til hliðar. Til að útfæra lookAt fallið fyrir mouse look settir ég eye sem staðsetninguna á notandanum og reiknaði saman með cos og sin hvert notandinn var að horfa á. Útfærslan er hér.

```
var eye = vec3(userX, userY, 5);
var dir = vec3(
  Math.cos(yaw)*Math.cos(pitch),
  Math.sin(yaw) * Math.cos(pitch),
  Math.sin(pitch)
);
mv = lookAt(eye, add(eye, dir), vec3(0.0, 0.0, 1.0));
```

```
//Main car
carDirection += 0.5;
if (carDirection > 360.0) carDirection = 0.0;
carXPos = (TRACK_RADIUS - 6) * Math.sin(radians(carDirection));
carYPos = (TRACK_RADIUS - 6) * Math.cos(radians(carDirection));
//Second car
car2Angle += car2Speed;
if (car2Angle < 1) car2Angle = 360;
car2X = car2Radius * Math.sin(radians(car2Angle));
car2Y = car2Radius * Math.cos(radians(car2Angle));
```

```
//Movement for the mouse look
case 87: // W
  userX += Math.cos(yaw) * speed;
  userY += Math.sin(yaw) * speed;
  break;

case 83: // S
  userX -= Math.cos(yaw) * speed;
  userY -= Math.sin(yaw) * speed;
  break;

case 65: // A
  userX += -Math.sin(yaw) * speed;
  userY += Math.cos(yaw) * speed;
  break;

case 68: // D
  userX += Math.sin(yaw) * speed;
  userY += -Math.cos(yaw) * speed;
  break;
```

Síðasta viðbótin var flugvélin, ég teiknaði hana úr 3 teningum, einn fyrir búkinn, einn fyrir vængina sem fer í gegnum búkinn til að búa til 2 vængi og svo einn fyrir stélið. Breyturnar sem ég gerði voru fyrir hraðan, hæðina, stærðina á eight curve og T til að reikna staðsetninguna. Hreyfingin á vélinni útfærði ég í sér falli, planeMovement, þar sem reiknað er flugið samkvæmt eight curve og einnig er útfært stefnuna á vélinni sem ég fékk hjálp frá chatGPT til að útfæra. Það er síðan kallað á þessu föll inn í drawScenery fallinu þar sem drawPlane notar útkomuna á planeMovement til að teikna vélina.

Í verkefninum notaði ég chatGPT til að hjálpa mér að skilja hvernig ætti að útfæra hlutina t.d. hvernig ætti að láta annan bílinn keyra í öfuga átt og hvernig flugvélin á að snúa í hverri teikningu. Einnig notaði ég chatGPT til að útfæra

```
function planeMovement(mv) {
  planeT += planeSpeed;

  var x = planeA * Math.sin(planeT);
  var y = planeA * Math.sin(planeT) * Math.cos(planeT);

  var dx = Math.cos(planeT);
  var dy = Math.cos(2.0 * planeT);
  var headingDeg = Math.atan2(dy, dx) * 180.0 / Math.PI;

  var mvPlane = mult(mv, translate(x, y, planeHeight));
  mvPlane = mult(mvPlane, rotateZ(headingDeg));
  return mvPlane;
}
```

litina á öllu í verkefninu, húsin, brúnna flugvélina og einnig bakgrunninn. Mynd af heildarútkomunni er hér fyrir neðan.

Sláið á 0, 1, ..., 8 til að skipta um sjónarhornr

Örvalyklar hækka og lækka staðsetningu áhorfanda

